

FRONTEND ASSIGNMENT-2

Ques1: What is CSS Box Model?

Ans1: The CSS Box Model describes how every HTML element is represented as a rectangular box.

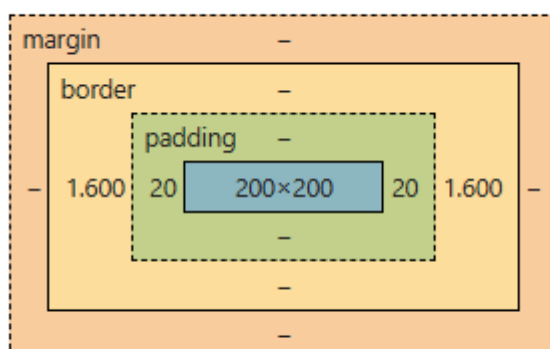
It consists of four parts:

1. **Content** – The actual text, image, or other content.
2. **Padding** – Space between the content and border.
3. **Border** – A line surrounding the padding and content.
4. **Margin** – Space outside the border between elements.

Example:

```
div{  
  width: 200px;  
  padding: 20px;  
  border: 5px solid black;  
  margin: 10px;  
}
```

The box can be visualized as:



Ques2: What is Media Query? Explain with sample code?

Ans2: A media query is a CSS technique used to apply different styles depending on the characteristics of the device or screen, such as its width.

It is commonly used to create responsive websites.

Example:

```
body {  
    background-color: lightblue;  
}
```

```
@media screen and (max-width: 600px) {  
    body {  
        background-color: lightgreen;  
    }  
}
```

Here, the background is light blue on screens wider than 600px and light green when the screen width is 600px or less.

Ques3: What are the features of Bootstrap?

Ans3: Bootstrap is a popular front-end framework used for developing responsive and mobile-first websites.

Important features include:

- Responsive design
- Mobile-first approach
- Grid system
- Predefined CSS classes
- Ready-made components
- Buttons, cards, forms, navbar, etc.
- JavaScript components
- Easy customization
- Cross-browser compatibility
- Utility classes for spacing, colors, display, positioning, etc.

Ques4: Write an example of basic grid structure in Bootstrap?

Ans4: Bootstrap provides a 12-column grid system.

Basic structure:

```
<!DOCTYPE html>

<html>

<head>

  <title>Bootstrap Grid</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">

</head>

<body>

<div class="container">

  <div class="row">

<div class="col-md-4">

  Column 1

</div>

<div class="col-md-4">

  Column 2

</div>

<div class="col-md-4">

  Column 3

</div>

</div>

</div>

</body>

</html>
```

Here:

- `.container` creates a responsive container.

- .row creates a row.
- .col-md-4 takes 4 out of 12 columns.
- Three .col-md-4 columns together occupy the complete row.

Ques5: What are CSS Selectors?

Ans5: CSS selectors are patterns used to select HTML elements to which CSS styles should be applied.

Common CSS selectors are:

1. Element Selector

Selects all elements of a particular type.

```
p{
    color: blue;
}
```

2. ID Selector

Selects an element using its id.

```
#heading{
    color: red;
}
```

3. Class Selector

Selects elements using their class.

```
.box{
    background-color: yellow;
}
```

4. Universal Selector

Selects all HTML elements.

```
*{
    margin: 0;
}
```

5. Group Selector

Applies the same style to multiple elements.

```
h1, h2, p {  
    font-family: Arial;  
}
```

Ques6: What are CSS Backgrounds? List the properties?

Ans6: CSS backgrounds are used to set the background appearance of HTML elements.

Important background properties include:

1. background-color
2. background-image
3. background-repeat
4. background-position
5. background-size
6. background-attachment
7. background-origin
8. background-clip
9. background

Example:

```
body {  
    background-color: lightblue;  
    background-image: url("image.jpg");  
    background-repeat: no-repeat;  
    background-position: center;  
    background-size: cover;  
}
```

The shorthand property can also be used:

```
body {  
    background: lightblue url("image.jpg") no-repeat center/cover;  
}
```

Ques7: How can we overlap elements in CSS?

Ans7: Elements can be overlapped using positioning and the z-index property.

For example:

```
<div class="box1"></div>
```

```
<div class="box2"></div>
```

```
.box1 {  
  width: 150px;  
  height: 150px;  
  background: red;  
  position: absolute;  
  top: 50px;  
  left: 50px;  
}
```

```
.box2 {  
  width: 150px;  
  height: 150px;  
  background: blue;  
  position: absolute;  
  top: 100px;  
  left: 100px;  
}
```

```
.box1 {  
  z-index: 1;  
}
```

```
.box2 {  
  z-index: 2;  
}
```

Ques8: What are the different positioning properties in CSS?

Ans8: CSS has five main values for the position property:

1. Static

Default position of an element.

```
div {  
  position: static;  
}
```

2. Relative

The element remains in the normal document flow but can be moved relative to its original position.

```
div {  
  position: relative;  
  top: 20px;  
  left: 10px;  
}
```

3. Absolute

The element is positioned relative to its nearest positioned ancestor.

```
div {  
  position: absolute;  
  top: 20px;  
  right: 10px;  
}
```

4. Fixed

The element remains fixed relative to the browser viewport, even when the page is scrolled.

```
div {
```

```
position: fixed;

bottom: 10px;

right: 10px;

}
```

5. Sticky

The element behaves like a relatively positioned element until a specified scrolling position is reached, after which it sticks.

```
div {

    position: sticky;

    top: 0;

}
```

Ques9: What is ECMAScript?

Ans9: ECMAScript is a standardized scripting language specification that defines the rules and features of scripting languages such as JavaScript.

JavaScript follows the ECMAScript standard.

ECMAScript defines features such as:

- Variables
- Data types
- Functions
- Objects
- Classes
- Operators
- Loops
- Modules
- Modern JavaScript features

In simple words:

ECMAScript is the standard/specification, while JavaScript is an implementation of that standard.

Ques10: Difference between var, let and const. Explain Temporal Dead Zone?

Ans10: Difference between var, let, and const

Feature	var	let	const
Scope	Function scope	Block scope	Block scope
Redeclaration	Allowed	Not allowed in same scope	Not allowed in same scope
Reassignment	Allowed	Allowed	Not allowed
Hoisting	Yes	Yes	Yes
TDZ	No	Yes	Yes

Var

```
var x = 10;  
x = 20;  
console.log(x);
```

Output:

20

Let

```
let x = 10;  
x = 20;  
console.log(x);
```

Output:

20

Const

```
const x = 10;  
// x = 20; // Error
```

Temporal Dead Zone (TDZ)

The Temporal Dead Zone is the period between entering a block and the point where a let or const variable is initialized.

Example:

```
console.log(x);
```

```
let x = 10;
```

This produces a `ReferenceError` because `x` is in the Temporal Dead Zone before its declaration is executed.